National University of Sciences and Technology (NUST) · Faculty of Computing
EE-353 Computer Networks · BESE-16AB · Stage 1: Project Proposal

Learning the Cost of Rerouting: ML-Based Disruption-Aware Elephant-Flow Load Balancing in SDN

Instructor: Ali Hassan · Lab Engineer: Rafia Ahmad

Name	Section	CMS ID
Mutahar	BESE-16AB	_____
Uswa Fatima	BESE-16AB	_____

Domain: Software-Defined Networking (SDN): Traffic Engineering and Load Balancing

Summary. When a software-defined network (SDN) controller moves a large flow to a less busy path, the move itself slows that flow down. We will train a small machine-learning model to predict this slowdown from statistics the controller already collects, so the controller moves a flow only when the move is worth it. We will implement the method in Mininet and Ryu and compare it against ECMP and Hedera-style rerouting.

1. Problem Description

Data-center and campus networks provide several parallel paths between any two machines. The standard way to spread traffic over them, Equal-Cost Multi-Path (ECMP) routing, hashes each flow onto a path and never looks at how busy the paths are [1]. Most flows are small, but a few large, long-lived flows (*elephant flows*) carry most of the bytes. When two elephants hash onto the same path, that path saturates while a parallel path stays nearly idle, and every small flow sharing the saturated path is delayed as well [1, 2].

An SDN controller can detect the hot path and move one elephant to an idle path [1]. However, moving a flow in the middle of a transfer is not free. Packets on the old and new paths arrive out of order, TCP interprets this as loss and cuts its sending rate, and a controller that reacts too eagerly can move flows back and forth [3, 4]. On an ONOS/Open vSwitch testbed, rerouting a TCP flow every 0.1 s cut its throughput to as low as 35% of normal, and every 2 s to about 60% [5]. **The problem we address: current SDN load balancers decide whether to move a flow without knowing how much the move will hurt that flow.**

2. Existing Approach and Identified Limitation

Existing approach to be improved. Hedera-style dynamic rerouting [1]: every few seconds the controller identifies flows above a rate threshold (10% of link capacity in Hedera) and moves them to the least-loaded path. Recent controllers follow the same pattern, including LBRouting [2], DPLBAnt [6] and the Streamlined Pathway (SP) balancer [7]. Static ECMP is the second baseline.

Identified limitation. These controllers decide using path load alone. Hedera models no migration cost, reordering or oscillation and does not count how many flows it moves [1]; the recent Ryu/Mininet work we reviewed reports throughput, utilization, loss and delay but not the damage that rerouting does to the moved flow [7, 6]. The idea of rerouting cautiously is known: Hermes moves a flow only when the new path is clearly better, citing TCP reordering and oscillation, but it runs on end hosts with fixed thresholds and reports only flow completion time [3]. FlexDATE treats network disturbance as a metric, but through offline reinforcement learning and linear programming rather than a per-flow decision at reroute time [8]. **No work we found gives a reactive OpenFlow controller an explicit, per-flow estimate of rerouting damage, or measures that damage against ECMP and Hedera.**

3. Proposed Solution

We give the controller a way to predict the damage of a move before making it, learned from measurements rather than guessed. The work has four steps:

- **Build the baselines.** Implement ECMP (OpenFlow 1.3 select groups) and Hedera-style rerouting [1] in a Ryu controller on a Mininet network with several parallel paths.
- **Measure what moves cost.** Deliberately reroute many flows under varied conditions. For each move, record what the controller could see beforehand (flow rate, bytes sent, delay difference between old and new path, load on both paths, time since the flow last moved) and the damage that followed (retransmitted and out-of-order segments, and the throughput drop over the next few seconds). Path delay and repeated moves are expected to

matter most [5].

- **Train a small model.** A decision tree or gradient-boosted model (scikit-learn) learns to predict a move's damage from those inputs. Its inference takes well under a millisecond, unlike deep reinforcement learning approaches that need thousands of training episodes and add inference delay [9].
- **Use it in the decision.** The controller moves a flow only if the expected benefit exceeds the predicted damage, and never moves the same flow again within a cooldown period:

$$\text{move } f \text{ from } p \text{ to } q \quad \text{if} \quad (\hat{U}_p - \hat{U}_q) \cdot r_f > \hat{C}_{ML}(f, p, q) \quad \text{and} \quad t - t_{last}(f) > T_{cool}$$

Here $\hat{U}$ is the smoothed (EWMA) load of a path, following low-overhead SDN monitoring practice [10]; r_f is the flow's rate; $\hat{C}_{ML}$ is the model's predicted damage; and T_{cool} is the cooldown. To isolate the value of machine learning, we also build a variant that uses a hand-written damage formula in place of the model. New paths are installed before old rules are removed, in the spirit of consistent network updates [11], so the damage we measure comes from TCP behaviour rather than packets lost during the switch-over.

4. Literature Review

Recent SDN load balancing work falls into two lines. **Heuristic elephant-flow rerouting** detects large flows and moves them off congested links. Tang et al. reduce the sampling cost of elephant detection and route elephants and mice differently [2]. DPLBAnt uses ant-colony optimization to choose reroute paths and reports 3.5× the throughput of ECMP [6]. Al-Saadi et al. classify flows with unsupervised learning before path selection [12]. The SP balancer combines proactive ECMP with controller scheduling in Mininet and Ryu and reports about 14% more bisection bandwidth than Hedera and ECMP [7]. **Learning-based routing** trains deep Q-networks or safe reinforcement-learning agents to choose paths [13, 14]; a 2025 review finds these need 1,000-5,000 training episodes, add inference latency and mostly remain in simulation [9].

Across both lines, success is measured by throughput, delay, loss and utilization. Surveys note recurring gaps: few topologies, little testing under topology changes, and limited metrics [15], while asymmetric topologies and partial link failures are common in practice [16]. The cost of the reroute itself has been studied at end hosts [3, 4], in offline traffic engineering [8], and as a measurement study [5], but not as a learned, per-flow decision rule inside an SDN controller. Table 1 compares the most relevant works.

Paper	Year	Problem	Existing Algorithm	Dataset / Environment	Key Metric	Main Limitation
Alawadi, SP [7]	2026	Elephant collisions under ECMP	ECMP, Hedera, Sieve	Mininet + Ryu, k=4 fat-tree, 10 Mbps	Bisection bandwidth, utilization, loss, RTT	Fat-tree only; threshold detection imprecise; no disruption metrics
Goteti & Reddy [9]	2025	DQL routing in SDN (review)	ECMP, Dijkstra	Mostly Mininet + Ryu	Throughput, delay, FCT	1,000-5,000 training episodes; added inference latency
Xie [14]	2025	Static routing under bursty load	Static routing, load balancing	Not specified	Throughput, delay, loss	Environment unclear; training cost
Dinh et al. [13]	2024	Unsafe DRL exploration in load balancing	DRL, non-RL baselines	Flow- and packet-based simulators	End-to-end delay, constraint violations	Needs training and fine-tuning; simulation only
Al-Saadi et al. [12]	2023	Same paths for elephants and mice	Default Ryu routing	Several data-center topologies	Throughput, bandwidth	Better in about 70% of cases; no migration cost
Ye et al., FlexDATE [8]	2022	Rerouting disturbs many flows	ECMP, full-reroute TE	SDN TE with RL + LP	Load balance vs optimal, disturbance	Offline RL/LP; not a reactive per-flow rule
Hamdan et al., DPLBAnt [6]	2021	Fixed thresholds, hop-count rerouting	ECMP, FlowSeer	Simulation	Delay, throughput, loss	Reroute frequency and reordering not reported
Tang et al., LBRouting [2]	2021	Costly elephant detection	Sampling-based detection	SDN experiments	Detection accuracy, sampled packets	Disruption from moving flows not evaluated
Zhang et al., Hermes [3]	2017	Reordering and oscillation from rerouting	ECMP, CONGA, LetFlow, Presto	Testbed + simulation, end hosts	Flow completion time	Host-based, fixed margins; damage not reported
Cârpa et al. [5]	2017	Effect of frequent reroutes on TCP	Periodic SDN rerouting	ONOS + OVS testbed, TCP Cubic	TCP throughput after reroutes	Measures harm; proposes no decision rule

Table 1. Comparison of SDN load-balancing approaches. Limitations for [2, 6, 12, 13, 14] are taken from the published abstracts and will be confirmed against the full texts.

5. Planned Experiments

We compare four controllers on identical traffic: (i) ECMP, (ii) Hedera-style rerouting [1], (iii) our rule with a hand-written damage formula, and (iv) our rule with the ML damage model.

- **Networks:** a four-path leaf-spine with unequal path delays (so that moves cause real reordering), a k=4 fat-tree as used in prior work [7], and a scenario in which one link fails or degrades mid-run [16].
- **Traffic:** random elephants mixed with many short flows at 30-90% offered load, plus an AI-training pattern of synchronized rounds of equal-size elephant flows, where each round finishes only when its slowest flow does.
- **Generalization:** the model is trained on some topologies and loads and tested on unseen ones.
- **Rigor:** 10 runs per configuration, reported as means with 95% confidence intervals.
- **Optional (Stage 3):** repeat the main experiment on a small Raspberry Pi testbed running Open vSwitch to confirm the result on physical switches.

6. Evaluation Metrics

- **Benefit kept:** elephant throughput, AI-round completion time, short-flow completion time (median and 99th percentile), and maximum link utilization.
- **Damage reduced:** retransmitted and out-of-order segments after each move, throughput drop after each move, reroutes per minute, and back-and-forth moves (flaps) [5, 3].
- **Model quality:** prediction error on unseen scenarios and decision time per flow.

7. Expected Outcomes

We expect our controller to retain most of Hedera's gain over ECMP while clearly reducing the damage caused by rerouting, and the learned model to outperform the hand-written formula. If moves turn out to cause little damage, or the formula performs as well as the model, we will report this as measured. By the Stage 2 milestone (15 November 2026) both baselines will be implemented and tested, with the first damage measurements and dataset.

8. Required Software, Tools and Resources

Tool	Purpose
Ubuntu Linux (native)	Host operating system for all experiments
Mininet + Open vSwitch	Network emulation; OpenFlow 1.3 group tables for ECMP
Ryu controller (os-ken fork [17] as fallback)	Python SDN controllers for baselines and the proposed method
scikit-learn	Training and evaluating the damage-prediction model
iperf3, tcpdump, Wireshark	Traffic generation; counting retransmitted and out-of-order segments
Python (pandas, matplotlib)	Experiment automation, statistics and plots
Raspberry Pi 4/5 (optional)	Small physical testbed, offered by the instructor
GitHub	Source code, scripts and README for reproducibility

References

[1] M. Al-Fares, S. Radhakrishnan, B. Raghavan, N. Huang and A. Vahdat, "Hedera: Dynamic flow scheduling for data center networks," in *Proc. USENIX NSDI*, 2010. Available: https://raghavan.usc.edu/papers/hedera-nsdi10.pdf

[2] F. Tang, H. Zhang, L. T. Yang and L. Chen, "Elephant flow detection and load-balanced routing with efficient sampling and classification," *IEEE Trans. Cloud Computing*, vol. 9, no. 3, pp. 1022-1036, 2021. Available: https://pure.ecnu.edu.cn/en/publications/elephant-flow-detection-and-load-balanced-routing-with-efficient-/

[3] H. Zhang, J. Zhang, W. Bai, K. Chen and M. Chowdhury, "Resilient datacenter load balancing in the wild," in *Proc. ACM SIGCOMM*, 2017. Available: https://cse.hkust.edu.hk/~kaichen/papers/hermes-sigcomm17.pdf

[4] A. Kabbani, B. Vamanan, J. Hasan and F. Duchene, "FlowBender: Flow-level adaptive routing for improved latency and throughput in datacenter networks," in *Proc. ACM CoNEXT*, 2014. Available: https://conferences2.sigcomm.org/co-next/2014/CoNEXT_papers/p149.pdf

[5] R. Cârpa, M. Dias de Assunção, O. Glück, L. Lefèvre and J.-C. Mignot, "Evaluating the impact of SDN-induced frequent route changes on TCP flows," in *Proc. Int. Conf. Network and Service Management (CNSM)*, 2017. Available: https://perso.ens-lyon.fr/laurent.lefevre/pdf/CNSM2017_Carpa_Assuncao_Gluck_Lefevre_Mignot.pdf

[6] M. Hamdan, S. Khan, A. Abdelaziz, S. Sadiah, N. Shaikh-Husin, S. Al Otaibi, C. Maple and M. N. Marsono, "DPLBAnt: Improved load balancing technique based on detection and rerouting of elephant flows in software-defined networks," *Computer Communications*, vol. 180, pp. 315-327, 2021. Available: https://pure.kfupm.edu.sa/en/publications/dplbant-improved-load-balancing-technique-based-on-detection-and-/

[7] A. H. Alawadi, "Streamlined Pathway (SP) approach: An efficient load balancer to enhance quality of service," arXiv:2601.08887, 2026. Available: https://arxiv.org/pdf/2601.08887

[8] M. Ye, J. Zhang, Z. Guo and H. J. Chao, "FlexDATE: Flexible and disturbance-aware traffic engineering with reinforcement learning in software-defined networks," *IEEE/ACM Trans. Networking*, 2022, doi: 10.1109/TNET.2022.3217083. Available: https://wp.nyu.edu/minghao/flexdate/

[9] D. Goteti and V. K. Reddy, "AI-driven routing pipeline in software-defined networks using DQL: A mini review," *Frontiers in Artificial Intelligence*, 2025, doi: 10.3389/frai.2025.1685155.

[10] S. R. Chowdhury, M. F. Bari, R. Ahmed and R. Boutaba, "PayLess: A low cost network monitoring framework for software defined networks," in *Proc. IEEE/IFIP NOMS*, 2014. Available: https://rboutaba.cs.uwaterloo.ca/Papers/Conferences/2014/ShihaburNOMS14.pdf

[11] X. Jin, H. H. Liu, R. Gandhi, S. Kandula, R. Mahajan, M. Zhang, J. Rexford and R. Wattenhofer, "Dynamic scheduling of network updates," in *Proc. ACM SIGCOMM*, 2014. Available: https://www.cs.princeton.edu/~jrex/papers/dionysus14.pdf

[12] M. Al-Saadi, A. Khan, V. Kelefouras, D. J. Walker and B. Al-Saadi, "SDN-based routing framework for elephant and mice flows using unsupervised machine learning," *Network* (MDPI), vol. 3, no. 1, pp. 218-238, 2023, doi: 10.3390/network3010011.

[13] L. Dinh, P. T. A. Quang and J. Leguay, "Safe load balancing in software-defined-networking," arXiv:2410.16846, 2024. Available: https://papers.cool/arxiv/2410.16846

[14] B. Xie, "Deep reinforcement learning-based intelligent traffic scheduling in software-defined networks," *Informatica*, vol. 49, no. 22, 2025. Available: https://www.informatica.si/index.php/informatica/article/view/8109

[15] A. H. Alhilali and A. Montazerolghaem, "Artificial intelligence based load balancing in SDN: A comprehensive survey," arXiv:2308.02149, 2023. Available: https://arxiv.org/pdf/2308.02149

[16] H. R. Rehman, S. Mahmood, S. M. Irteza, F. R. Dogar and I. A. Qazi, "Efficient datacenter load balancing with microslices," in *Proc. IFIP Networking*, 2025. Available: https://networking.ifip.org/2025/images/Net25_papers/1571127919.pdf

[17] OpenStack, "os-ken: A component-based software defined networking framework" (maintained fork of Ryu), Python Package Index, 2026. Available: https://pypi.org/project/os-ken